70. Операторы break и continue

Хотя вы уже видели оператор break в связке с **оператором switch**, все же он заслуживает большего внимания, поскольку может использоваться и с циклами. **Оператор break** приводит к завершению выполнения циклов do, for или while.

Оглавление:

1. break и switch
2. break и циклы
3. break и return
4. Оператор continue
5. break и continue

break и switch

В контексте оператора switch оператор break обычно используется в конце каждого кейса для его завершения (предотвращая **fall-through**):

```
switch (op)
{
    case '+'
        doAddition(a, b);
        break;
    case '-'
        doSubtraction(a, b);
        break;
    case '*'
        doMultiplication(a, b);
        break;
    case '/'
        doDivision(a, b);
        break;
}
```

break и циклы

В контексте циклов оператор break используется для завершения работы цикла раньше времени:

```
#include <iostream>

int main()
{
    int sum = 0;

    // Разрешаем пользователю ввести до 10 чисел
    for (int count=0; count < 10; ++count)
    {
        std::cout << "Enter a number to add, or 0 to exit: ";
        int val;
        std::cin >> val;

        // Выходим из цикла, если пользователь ввел 0
        if (val == 0)
            break;

        // В противном случае, добавляем число к общей сумме
        sum += val;
    }

    std::cout << "The sum of all the numbers you entered is " << sum << "\n";

    return 0;
}
```

Эта программа позволяет пользователю ввести до 10 чисел и в конце подсчитывает их сумму. Если пользователь введет `0`, то выполнится break и цикл завершится (не важно, сколько чисел в этот момент успел ввести пользователь).

Обратите внимание, оператор break может использоваться и для выхода из бесконечного цикла:

```
#include <iostream>

int main()
{
    while (true) // бесконечный цикл
    {
        std::cout << "Enter 0 to exit or anything else to continue: ";
        int val;
        std::cin >> val;

        // Выходим из цикла, если пользователь ввел 0
        if (val == 0)
            break;
    }

    std::cout << "We're out!\n";

    return 0;
}
```

break и return

Новички часто путают или не понимают разницы между операторами **break** и **return**. Оператор break завершает работу switch или цикла, а выполнение кода продолжается с первого стейтмента, который находится сразу же после этого switch или цикла. Оператор return завершает выполнение всей функции, в которой находится цикл, а выполнение продолжается в точке после вызова функции:

```
#include <iostream>

int breakOrReturn()
{
    while (true) // бесконечный цикл
    {
        std::cout << "Enter 'b' to break or 'r' to return: ";
        char sm;
        std::cin >> sm;

        if (sm == 'b')
            break; // выполнение кода продолжится с первого стейтмента после цикла

        if (sm == 'r')
            return 1; // выполнение return приводит к тому, что управление сразу возвратится в caller (в этом случае, в функци

    // Использование оператора break приводит к тому, что выполнение цикла продолжится здесь

    std::cout << "We broke out of the loop\n";

    return 0;
}

int main()
{
    int returnValue = breakOrReturn();
    std::cout << "Function breakOrContinue returned " << returnValue << "\n";

    return 0;
}
```

Оператор continue

Оператор continue позволяет сразу перейти в конец тела цикла, пропуская весь код, который находится под ним. Это полезно в тех случаях, когда мы хотим завершить текущую итерацию раньше времени. Например:

```
#include <iostream>

int main()
{
    for (int count = 0; count < 20; ++count)
    {
        // Если число не делится нацело на 4, то пропускаем весь код в этой итерации после continue
        if ((count % 4) == 0)
            continue; // пропускаем всё и переходим в конец тела цикла

        // Если число не делится нацело на 4, то выполнение кода продолжается
        std::cout << count << std::endl;

        // Точка выполнения после оператора continue перемещается сюда
    }

    return 0;
}
```

Эта программа выведет все числа от 0 до 19, которые не делятся нацело на 4.

В случае с **циклом for** часть инкремента/декремента счетчика по-прежнему выполняется даже после выполнения continue (так как **инкремент/декремент** происходит вне тела цикла).

Будьте осторожны при использовании оператора continue с циклами **while** или **do while**. Поскольку в этих циклах инкремент счетчиков выполняется непосредственно в теле цикла, то использование continue может привести к тому, что цикл станет бесконечным! Например:

```
#include <iostream>

int main()
{
    int count(0);
    while (count < 10)
    {
        if (count == 5)
            continue; // переходим в конец тела цикла
        std::cout << count << " ";
        ++count;

        // Точка выполнения после оператора continue перемещается сюда
    }

    return 0;
}
```

Предполагается, что программа выведет все числа от 0 до 9, за исключением 5. Но на самом деле:

```
0 1 2 3 4
```

А затем цикл станет бесконечным. Когда значением `count` становится `5`, то условие **оператора if** станет true, затем выполнится continue и мы, минуя вывод числа и инкремент счетчика, перейдем к следующей итерации. Переменная `count` так и не увеличится. Как результат, в следующей итерации переменная `count` по-прежнему останется со значением `5`, а оператор if по-прежнему останется true, и цикл станет бесконечным.

А вот правильное решение, но с использованием цикла do while:

```
#include <iostream>

int main()
{
    int count(0);
    do
    {
        if (count == 5)
            continue; // переходим в конец тела цикла
        std::cout << count << " ";
        ++count;

        // Точка выполнения после оператора continue перемещается сюда
    } while (++count < 10); // этот код выполняется, так как он находится вне тела цикла

    return 0;
}
```

Результат выполнения программы:

```
0 1 2 3 4 6 7 8 9
```

break и continue

Многие учебники рекомендуют не использовать операторы break и continue, поскольку они приводят к произвольному перемещению точки выполнения программы по всему коду, что усложняет понимание и следование логике выполнения такого кода.

Тем не менее, разумное использование операторов break и continue может улучшить читабельность циклов в программе, уменьшив при этом количество вложенных блоков и необходимость наличия сложной логики выполнения циклов. Например, рассмотрим следующую программу:

```
#include <iostream>

int main()
{
    int count(0); // сначала количество итераций цикла
    bool exitloop(false); // контрольную завершение выполнения цикла
    while (!exitloop)
    {
        std::cout << "Enter 'e' to exit this loop or any other key to continue: ";
        char sm;
        std::cin >> sm;

        if (sm == 'e')
            exitloop = true;
        else
        {
            --count;
            std::cout << "We've iterated " << count << " times\n";
        }
    }

    return 0;
}
```

Эта программа использует **логическую переменную** для выхода из цикла, а также вложенный блок, который запускается только в том случае, если пользователь не использует символ выхода.

А вот более читабельная версия, но с использованием оператора break:

```
#include <iostream>

int main()
{
    int count(0); // сначала количество итераций цикла
    while (true) // выполнение цикла продолжается, если его не завершит пользователь
    {
        std::cout << "Enter 'e' to exit this loop or any other key to continue: ";
        char sm;
        std::cin >> sm;

        if (sm == 'e')
            break;

        --count;
        std::cout << "We've iterated " << count << " times\n";
    }

    return 0;
}
```

Здесь (с одним оператором break) мы избежали использования как логической переменной (а также понимания того, зачем она и где используется), так и оператора else с вложенным блоком.

Уменьшение количества используемых переменных и вложенных блоков улучшают читабельность и понимание кода намного больше, чем операторы break или continue могут нанести вред. По этой причине считается приемлемым их разумное использование.